

Hill Climbing & best first search

10:07 AM

Hill climbing is a simple heuristic search algorithm used to find the best solution by incrementally improving the current state until no further improvements can be made. The idea is to always move towards a state that is better (or "higher") than the current one, like climbing a hill until you reach the top.

Imagine you're blindfolded and standing on a hill. Your goal is to reach the highest point on the hill.

How Hill Climbing Works:

1. **Starting Point:**
 - You start at a random point on the hill.
2. **Evaluation:**
 - You feel the ground around you to determine which direction goes uphill (increases in height).
3. **Making a Move:**
 - You take a small step in the direction that seems to go the steepest uphill.
4. **Repeat:**
 - After each step, you check again to see which direction goes uphill and take another step in that direction.
 - You keep doing this until you find that all surrounding steps would take you downhill.
5. **Goal:**
 - You stop when you reach a point where no step in any direction goes uphill (this is the top of the hill, or a "peak").

Key Points:

- **Local Maximum:** In some cases, you might reach a peak that is the highest point in your immediate area, but not the highest point on the entire hill. This is called a "local maximum."
- **Global Maximum:** The highest point on the entire hill is the "global maximum." Hill climbing may sometimes get stuck at a local maximum if no higher points are nearby.

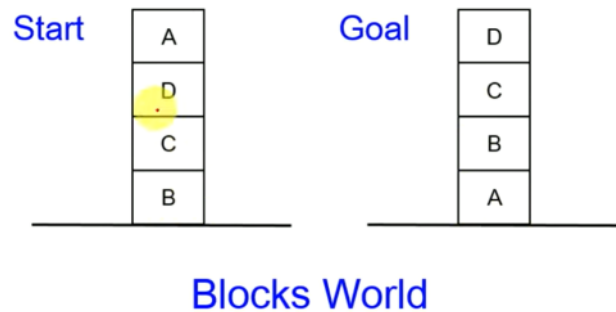
Example with Numbers:

Imagine you're trying to solve a very simple math problem: maximizing the value of $f(x) = -x^2 + 4x$

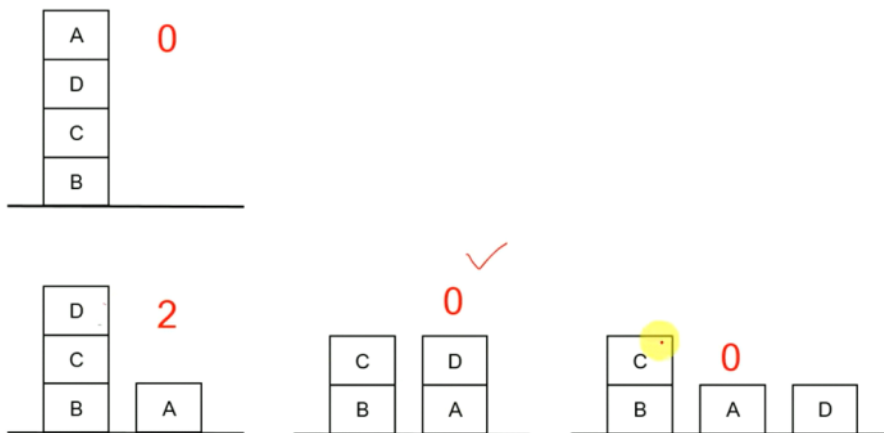
1. **Start at x=0:**
 - Calculate $f(0)$
 - Check nearby values, say $x=1$ and $x=-1$
2. **Move to x=1**
Calculate $f(1)=3$.
 - This is higher than $f(0)=0$, so you move to $x=1$.
3. **Next Step:**
 - Now check $x=2$ and $x=0$ (next possible moves).
 - $f(2)=4$, which is higher than $f(1)=3$, so you move to $x=2$.
4. **Keep Moving:**
 - At $x=2$, check $x=3$ and $x=1$.
 - $f(3)=3$, which is lower than $f(2)=4$, so you stay at $x=2$.
 - There are no higher values around, so you stop.

With Heuristic function 1

Example:



- +1 for each block that is resting on the thing it is supposed to be resting on.
- 1 for each block that is resting on a wrong thing.



We got stuck here with local maxima only

With Heuristic function-2

Feature	Hill Climbing	Steepest Ascent Hill Climbing
Neighbor Selection	First better neighbor found	Best neighbor with the highest improvement
Evaluation of Neighbors	Stops at first improvement	Evaluates all neighbors
Risk of Local Optima	Higher risk of getting stuck in local optima	Lower risk due to selecting the steepest ascent
Computational Efficiency	More efficient (faster)	Less efficient (slower) due to exhaustive search
Solution Quality	May result in suboptimal solutions	More likely to find better solutions

Best First Search

Best-First Search is an **informed search algorithm** that explores a search space by always expanding the **most promising node first**, based on an **evaluation (heuristic) function**. It uses **heuristic knowledge** to guide the search toward the goal efficiently.

Best First Search Algorithm:

1. Let OPEN be a priority queue containing the initial state.
2. Loop:
 - If OPEN is empty, return failure.
 - Node $\leftarrow$ Remove First(OPEN)
 - If Node is a Goal:
 - Return the Path from the initial state to Node.
 - Else:
 - Generate all successors of Node.
 - Put the newly generated nodes into OPEN according to their f values.
3. End Loop.

In the context of this algorithm:

- **OPEN** is the priority queue that holds nodes, ordered based on their heuristic values (in this case, straight-line distance).
- The algorithm repeatedly picks the most promising node (the one with the lowest heuristic value), expands it, and adds its successors to the OPEN list.
- The process continues until the goal is found or there are no more nodes to explore (indicating failure).

Key difference between Best First Search and Steepest Ascent Hill climbing

Best FS	Steepest HC
Key Characteristics: • Global Perspective: BFS keeps track of multiple paths and can backtrack if it finds that another path is more promising based on the heuristic. • Priority Queue: Nodes are stored in a priority queue, which orders nodes based on their heuristic values ($h(n)$). • Can Revisit Nodes: BFS may revisit nodes if it finds a potentially better path to them later. • Can Find the Optimal Path: Depending on the heuristic used, BFS can find the shortest or most efficient path to the goal.	• Local Perspective: It only considers the current state and its immediate neighbors, without maintaining a global view of the search space. • Greedy Approach: The algorithm always chooses the best immediate improvement, without considering whether this might lead to a suboptimal final solution. • No Backtracking: Once it moves to a new state, it does not return to previous states, which can cause it to get stuck in local optima. • Stops at Local Optimum: If no neighboring state offers a better value than the current state, the algorithm halts, which might result in a suboptimal solution.

